

COMS W4156 Advanced Software Engineering (ASE)

October 3, 2023



Agenda

1. issues with preliminary proposals
2. finish equivalence partitions and boundary analysis



Service **!** = App

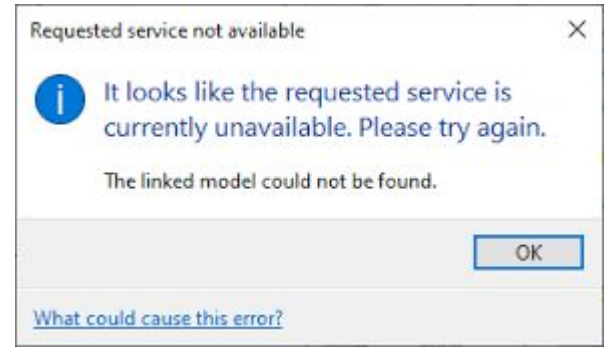
Many of the teams proposed apps, not services.
Apps will be added in the second iteration,
the first iteration is just services

Keep your app ideas for the second iteration!

If this issue applies to your team, please discuss with your Mentor how to revise your proposal to indeed propose a service

In some cases the proposed app is "close" to a plausible service and the proposal should not be hard to fix, in a few cases the problem is more fundamental

I'm going to repeat some slides from September 7, the second day of class. If you had not yet joined the class by then, or missed class that day, you should have watched the video

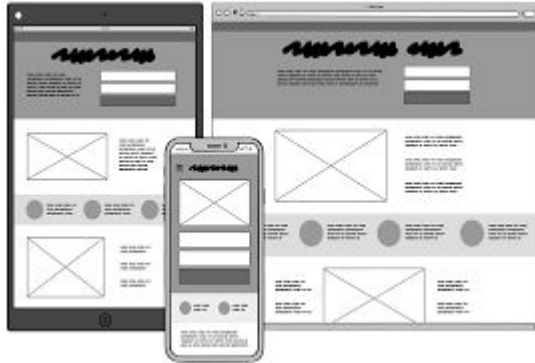


What is an App?

Software that does something for users

The developers of an app design a UI for users, often with a diagramming tool like draw.io

The internal design of the app code defines a set of interacting software units that together implement the UI



What is a Service?

Software that does something for other software

The developers of a service design an API for clients, often with a specification tool like [OpenAPI](https://openapi.org/)

The internal design of the service code defines a set of interacting software units that together implement the API



What is the Difference between a Service and an App?

If you ask google search, you will find a variety of contradictory answers



For the purposes of this course, an app (or application) is used by human users via a user interface (UI) whereas a service is used by code via an application programming interface (API)

Please do not say something like “our team’s service is an app that does xxx”, “the user does xxx with our service”, or “the user uses our service to do xxx”

If a human user is using it, then it is an app not a service

Instead say “our team’s service does xxx”, “the client does xxx with our service”, or “the client uses our service to do xxx”

Are there any Exceptions?



When I say human users, I mean human end-users - not human developers or administrators

A service may have a command line interface (CLI) or even a graphical user interface (GUI) for administrators to start/stop the service and perform other administrative actions, but it is not available to end-users

Developers test APIs with tools that mimic client code

- ★ For example, [Postman](#) and [SoapUI](#)

Developers test UIs with tools that mimic human end-users

- ★ For example, [Selenium](#) and [Cypress](#)



What about Software as a Service?

Software as a Service ([SaaS](#)) performs the service of supplying users with apps that are hosted on the cloud rather than being installed on the users' machines

But the human user still interacts with each app via a UI and code (that implements an app) interacts with the service via an API

A human user can use an app but cannot use a service, only clients can use a service



Non-SaaS Application



Application logic runs
on user's computer

SaaS Application



Application logic runs
in the cloud

Agenda

1. issues with preliminary proposals
2. finish equivalence partitions and boundary analysis



Baseline Testing Strategy

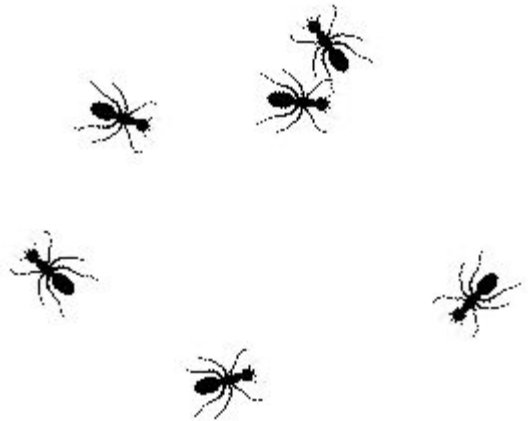
- *Test with one typical valid input, one atypical valid input, and one invalid input
[better: one invalid value right format, one invalid wrong format]*

It's better than no strategy

It's better than the common CS1 strategy -
test program with two or three typical valid inputs,
done

It applies to unit testing, integration testing,
system testing, API testing, ...

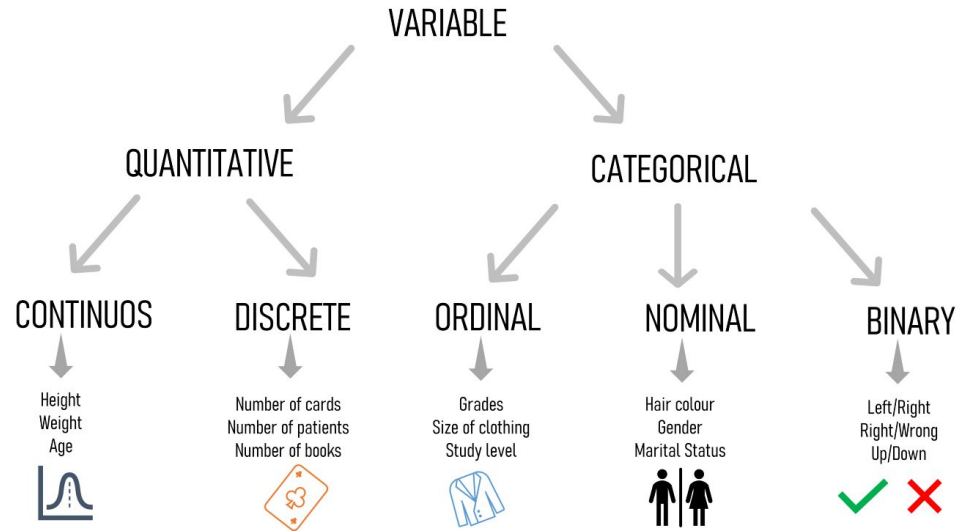
Think of it as just a starting point



Better Approaches to Choosing Test Inputs

If you have an automated coverage tool, or your program is small enough to check manually, try to achieve 100% coverage

Also test with a set of inputs that you think would make a nice demo - but keep in mind what might happen when the demo audience asks to “drive”. If this is higher level management in your company, your venture capital investor, or the instructor/TA who decides your grade, you cannot say No



How About Testing ALL Possible Inputs?

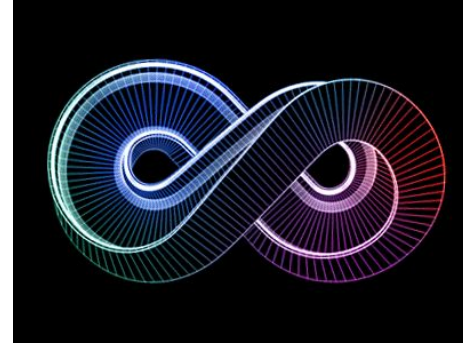
“Exhaustive Testing”

Usually “infinite” number of valid and invalid inputs
(assuming infinite size input buffers available)

Even if number of valid and invalid inputs is finite,
infeasible to try all of them

To maximize the chances of finding and fixing
bugs, we need to choose a practically small
number of inputs that, collectively,

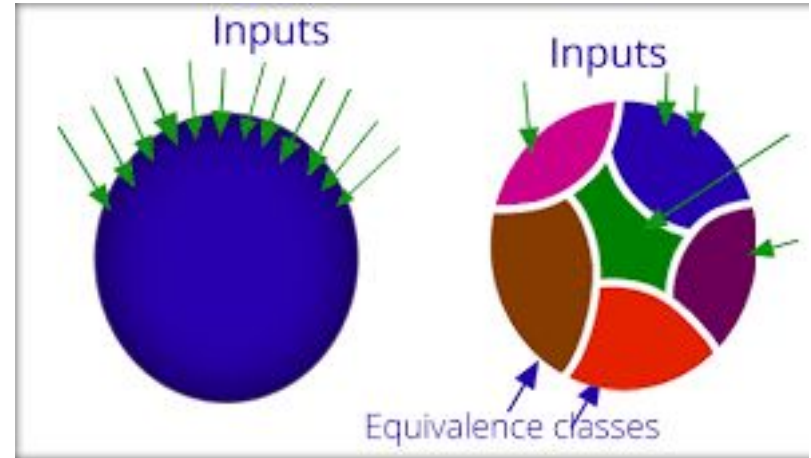
- represent all possible inputs and
- are *most likely* to reveal bugs



Equivalence Partitions

The simplest approach for representing all possible inputs to a system with a practically small number of examples is to divide the input space into *equivalence partitions* (also known as equivalence classes) based on **both**

- domain knowledge
- software engineering knowledge (where do bugs like to hide?)



Equivalence Partitions

Equivalence partition = set of alternative input values where the software can reasonably be expected to behave equivalently (similarly)

This does **not** mean literally the same output for every input in the class!



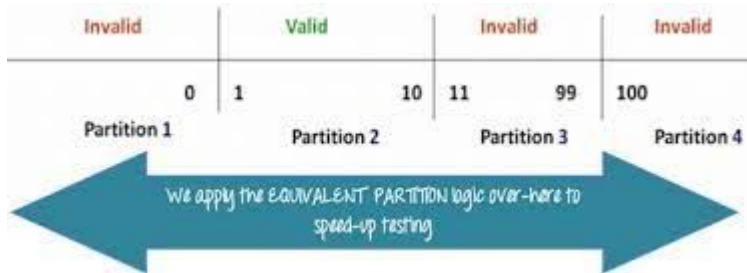
Instead it means that if the software behaves correctly on one example input chosen from that equivalence class, it is reasonable to believe it will behave correctly on all the other inputs in the same class

Think about what kind of computation or processing is needed for different members of the class - if it's not very similar then it's not actually an equivalence class and you need to rethink your partitioning strategy

Simple Example

If we are testing a “box” intended to accept numbers from 1 to 10, then do not write ten test cases for all 10 valid input numbers

If we are testing a “box” intended to accept numbers from 1 to 1000000, then do not write a million test cases for all 1000000 valid input numbers!



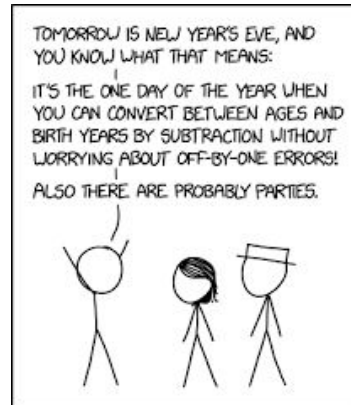
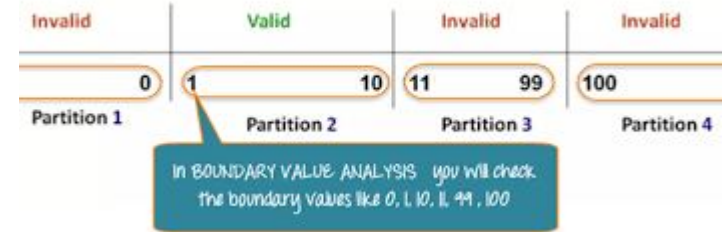
Instead, divide the test cases into three sets:

- 1) Equivalence partition with all valid inputs. Pick a single value from range 1 to 1000000 as a valid test case. If we select other values between 1 and 1000000 the result “should” also be correct (or also be incorrect)
- 2) Pick a single value for the partition with all values *below* the lower limit of the range, i.e., any value below 1
- 3) Pick a single value for the partition with all values *above* the upper limit, i.e., any value above 1000000

➤ Are we done yet?

Boundary Value Analysis (or just Boundary Analysis)

Equivalence partitions are typically extended with [boundary analysis](#) because input values at the extreme ends of the valid and invalid partitions are more likely to trigger bugs = [“off by one error”](#)

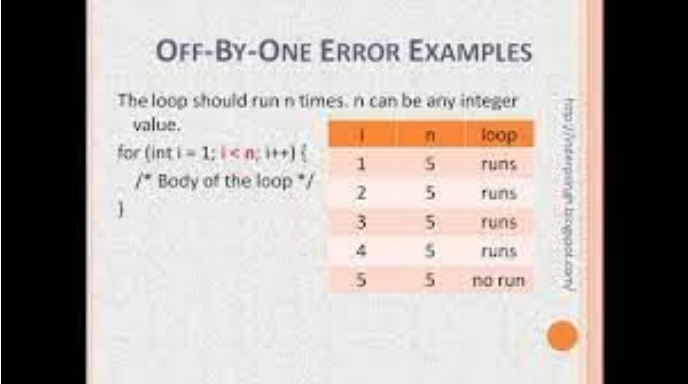


Boundary Value Analysis

- 1) Test cases with input exactly at the boundaries of the valid partition, i.e., values 1 and 1000000
- 2) Test data with values just below the extreme edges, i.e., values 0 and 999999
- 3) Test data with values just above the extreme edges, i.e., values 2 and 1000001

For integer domains, make sure to include zero, positive and negative examples, as well as the underlying primitive extremes

In this case, we already have 0 and some positive values, so we need to add some negative value, e.g., -1, plus MinInt and MaxInt



The slide is titled "OFF-BY-ONE ERROR EXAMPLES". It contains a code snippet for a loop and a table showing the results of running the loop for different values of *i* and *n*.

The loop should run *n* times. *n* can be any integer value.

```
for (int i = 1; i < n; i++) {  
    /* Body of the loop */  
}
```

<i>i</i>	<i>n</i>	loop
1	5	runs
2	5	runs
3	5	runs
4	5	runs
5	5	no run

A URL is visible on the right side of the slide: <http://www.paulhogan.com/>

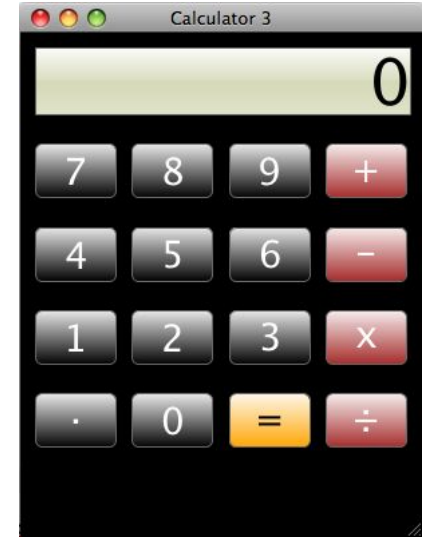
Example

Consider a calculator app implemented in software, with a GUI as shown. The user clicks the buttons with a mouse, but cannot enter text into the display

In mathematics, the “+” operation takes two numeric operands

Say this software is tested by an external testing team separate from the developers, so they don't know how the app was implemented

Based on [domain knowledge](#), a tester might reasonably expect the “+” operation to behave similarly on all pairs of numbers
< first operand, second operand >. Why?

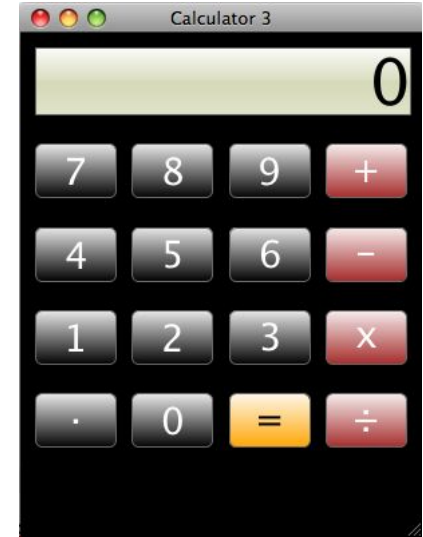


Continuing Example

So the tester tries an arbitrary pair of numbers:

< 3.141592653, 42 >

The calculator flashes 00000000. What might this mean?



Interpreting Test Outputs

Based on [software engineering knowledge](#), the tester might realize the flashing 00000000 could indicate:

- An error message because the software cannot handle numbers with more than N digits (e.g., 8)
- An error message because the software cannot handle mixed operands, i.e., one operand is floating point and the other is integer
- A bug
- Something else, e.g., the user has reached the limit of their “free trial” and now has to pay to continue using the calculator software



Continuing Calculator Example

Let's ignore the “.” key (planned for version 2.0) and assume the “+” operator is currently intended only for integer operands

A tester with software engineering knowledge, not just domain knowledge, might realize that a calculator implementation (or any other software dealing with integers) could behave differently for positive integers, zero, and negative integers

Thus they test with five different equivalence classes:

- < pos int, pos int >
- < pos int, neg int >
- < pos int, 0 >
- < neg int, neg int >
- < 0, 0 >

Is this sufficient?



Software is not Mathematics

Although the “+” operator is mathematically commutative, that does not mean that the implementation (which could be buggy) is commutative

Or, more generally, define equivalence partitions separately per parameter, and then extend to the cross-product

But notice the scaling problem with large numbers of parameters!

- $\langle \text{pos int}, \text{pos int} \rangle$
- $\langle \text{pos int}, \text{neg int} \rangle$
- $\langle \text{pos int}, 0 \rangle$
- $\langle \text{neg int}, \text{pos int} \rangle$
- $\langle \text{neg int}, \text{neg int} \rangle$
- $\langle \text{neg int}, 0 \rangle$
- $\langle 0, \text{pos int} \rangle$
- $\langle 0, \text{neg int} \rangle$
- $\langle 0, 0 \rangle$



Extremes

Let's say the calculator is intended to handle operands only up to 8 digits (or to any other fixed number of digits)

This means there is a MaxInt and a MinInt (negative), probably specific to what the calculator GUI was designed to display rather than MaxInt/MinInt of the programming language, compiler, operating system, etc.

What additional tests are needed?

- < xxx, MaxInt >
- < MaxInt, xxx >
- < xxx, MinInt >
- < MinInt, xxx >
- < MaxInt, MinInt >
- < MinInt, MaxInt >



Invalid Inputs to Calculator Example



What if the user could enter text into the display window in addition to mousing the keys

How does that change the set of equivalence partitions that need to be tested?

Another Example

What are the
equivalence partitions?

How would we test this
program?

Design-your-burger

Please make your selection

Burger:

Cooked: ☐ rare ☐ medium ☐ well

Cheese: ☐

Lettuce: ☐

Tomato: ☐

Onion: ☐

Ketchup: ☐

Mustard: ☐

Mayo: ☐

Secret sauce: ☐



[Nutrition facts](#) [About the chef](#)

Required vs Optional Inputs

Get

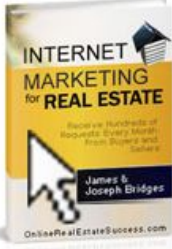
"Internet Marketing for Real Estate EBook"

via e-mail

Name*:

Email*:

Phone*:



* Indicates a required field

* Indicates an optional field

For some applications, equivalence classes need to address optional as well as required inputs, with or without default values

What are the equivalence partitions for this example?

How would we test this program?

Dependencies between Inputs

Distinct parameters may be related to each other in forming equivalence classes

Let's say we had a function that took two variables, blood sugar level and level of frustration, to calculate "mood"



Assume valid inputs are
Blood sugar 70 - 100
Frustration is 50% - 100%

For $50 \leq F \leq 90$

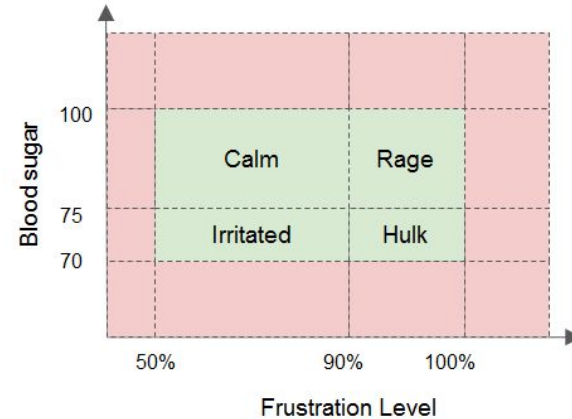
75 $\leq$ BS $\leq$ 100: Calm

BS < 75: Irritated

For $F > 90$

75 $\leq$ BS $\leq$ 100: Rage

BS < 75: [Hulk Smash](#)

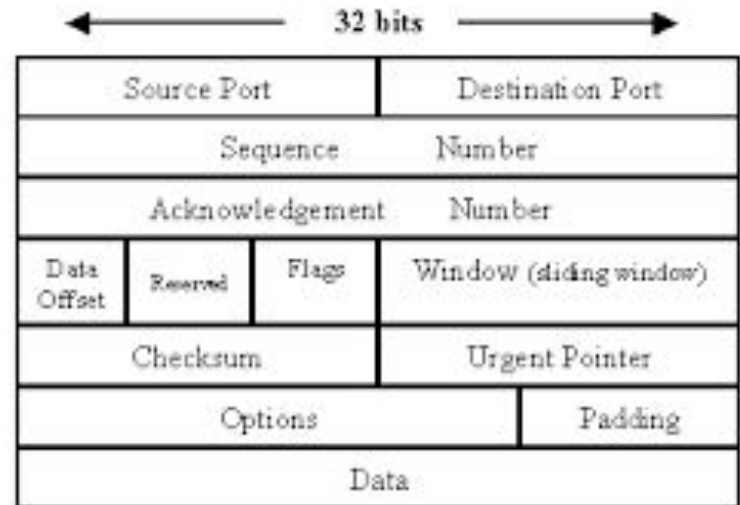


More Dependencies between Inputs

Equivalence classes may cut across multiple inputs:

Sometimes the value of one input restricts valid values for other inputs. For example, date of birth and age should be consistent (valid) or not (invalid)

Sometimes the value of one (small) input is intended as a sanity check on another (larger, more complicated) input, such as the [checksum in the network packet](#)

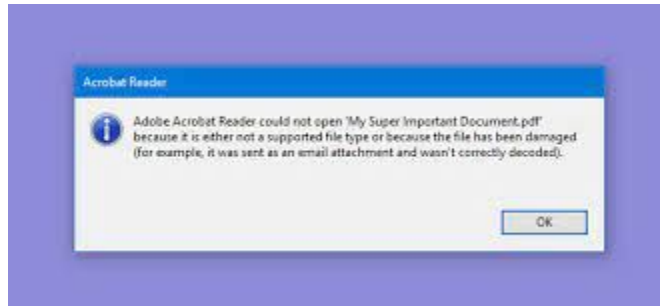


System to System Inputs

At the full system and service levels, inputs are often going to be numbers, strings (text), tuples (packets, json), and files

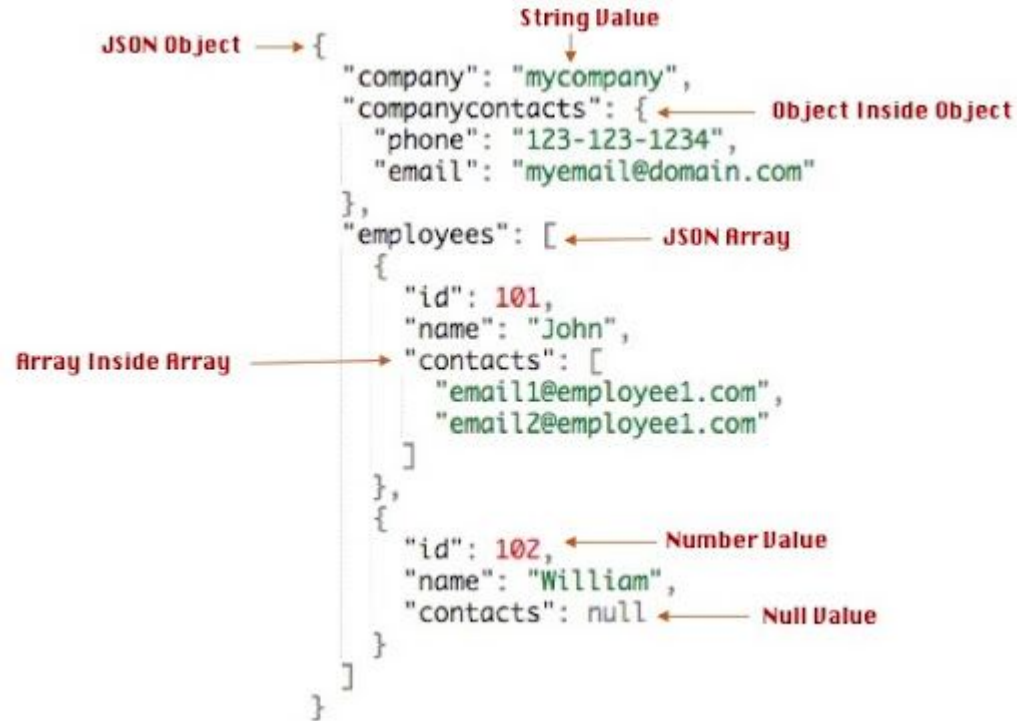
File inputs can:

- Exist or not
- Be below min or above max size
- Be readable, writeable, executable by current user (permissions)
- Correct, corrupted, garbled, etc. data and metadata formats - images, audio, video, ...



'Systems' here includes Services with APIs

What might be the equivalence classes for inputs with this format?



Unit Inputs

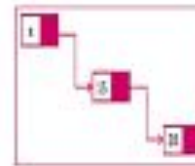
At the unit testing level, inputs/outputs can be arbitrary in-memory data structures, so equivalence classes can be more complex - and inputs more complicated to construct

Container data structures with content elements:

- In or not in container
- Empty or full container (or min/max number of elements)
- Duplicate elements may or may not be allowed
- Ordered or organized according to some metric, or not



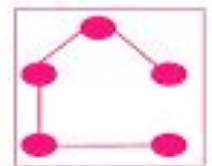
Sorting



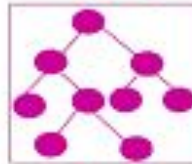
Link list



```
list
```



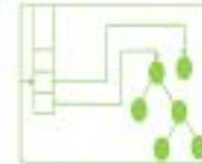
spanning tree



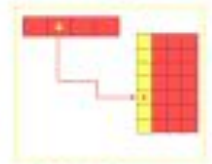
Tree



Graph

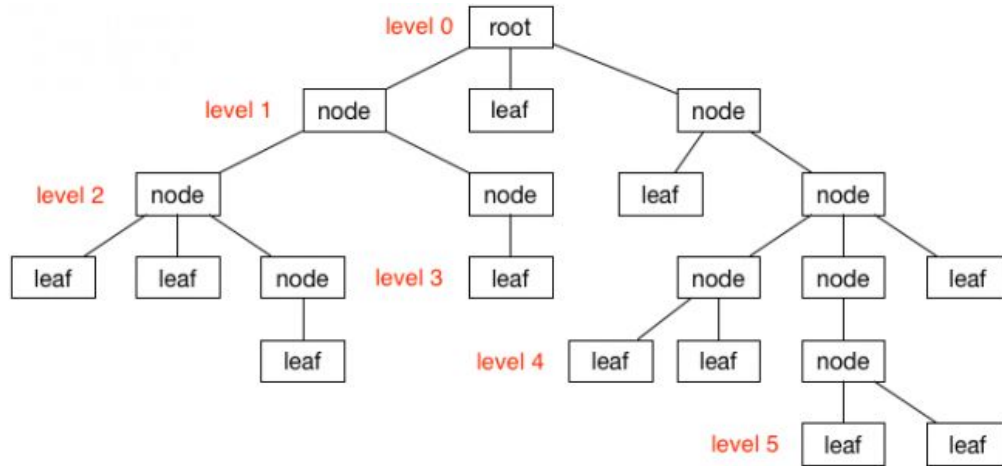


Stack



Hashing

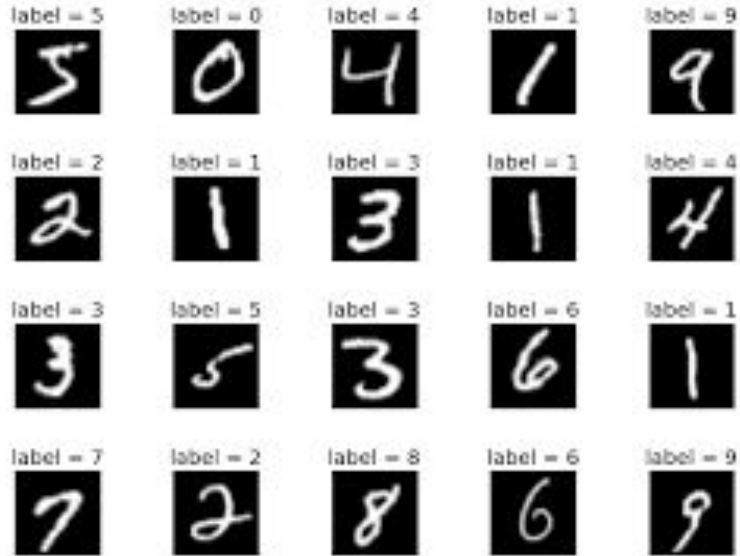
More Unit Inputs



Specific kinds of containers, e.g., tree - root node, interior node, leaf node, null tree, tree with exactly one node, “full” tree, balanced vs. unbalanced, various specialty trees - and graphs

How would test cases construct root, node and leaf objects to use as inputs?

Equivalence Partitions for Data Sets



Files containing “data sets” often correspond to a table (rows and columns)

This shows part of a well-known dataset for deep learning algorithms ([MNIST](#))

What are the equivalence classes?

Upcoming Assignments: Note extended deadlines!

- [Team Revised Project Proposal](#) due next Thursday, October 5, by 11:59pm
- [First Iteration](#) due Thursday, October 19, by 11:59pm
- [First Iteration Demo](#) due Thursday, October 26, by 11:59pm



Google Cloud Platform Credits

There are enough student coupons allocated to this class for every student to have one. You must use your .edu email address. If you have multiple .edu email addresses, please only use one. Contact your Mentor to get the URL to request your student coupon.



Thursday

unit testing with demo



Ask Me Anything